Python for computational Problem Solving · Unit_02 · Unit 2: Collections & Functions

DRAFT — UE26CS151A · Unit_02 — Unit 2: Collections & Functions · Complete Unit Master

Notes-first study master · source-cited lecture explanations and compact page takeaways

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the edited lesson notes by topic. Evidence chips point to transcript intervals and slide pages. The complete source transcripts, page text, formula candidates, and image locators are preserved in sibling JSON datasets. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · UE26CS151A · Unit_02: Unit 2: Collections & Functions
2. 2 · Lists and Tuples
3. 3 · Dictionary
4. 4 · Files
5. 5 · Function_Call_Definition_Variable_arguments
6. 6 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets
7. 7 · Source slide deck: JSON
8. 8 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args
9. 9 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args
10. Appendix A · Symbol glossary
11. Appendix B · Sources, coverage and gaps
12. Appendix C · Revision and retrieval practice

M1

Unit_02 · Unit 2: Collections & Functions

1 UE26CS151A · Unit_02: Unit 2: Collections & Functions

This unit master organizes source material for Python for computational Problem Solving, Unit_02, officially titled Unit 2: Collections & Functions. It includes 4 video transcripts and 8 source slide decks, with page-level text and rendered images where available. This draft preserves source wording and page order; it does not claim a complete human subject review or full audiovisual review. EV0000001

SHORT-NOTE RECAP

Unit map · UE26CS151A · Unit_02: Unit 2: Collections & Functions

Official catalogue entry: UE26CS151A, Unit_02 (Unit 2: Collections & Functions). Bucket folder present: yes. Videos: 4; slide decks: 8.

EV0000001

2 Lists and Tuples

Study focus: Lists and Tuples. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000002 · EV0000003

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Linear data structure where elements have a linear ordering and list has got some of the characteristics.

Key details

- a straightforward task, you would read three variables A B C and then each one of them have been assigned some value and then you will perform A plus B plus C and at the end you're going to be getting the sum.

- If I ask you to perform the sum of thousand numbers, the sum of thousand numbers.
- In this case, what you can do is very simple, where you can iterate over the loop or run over the loop a thousand times, and then try to find out the sum by updating the sum O variable.
- If we ask you to arrange these thousand elements in an ascending order.
- Fine, arranging the thousand elements in an ascending order.
- In this case, it becomes impossible for us to make use of thousand different variables and then perform the arranging operation.
- So in such case, the main reason is you need all the thousand items to be there at a same point of time because you are trying to do the comparison.
- it is always useful to have all the different elements under a similar name.
- Elements in a list are heterogeneous in nature, can be heterogeneous in nature.
- It could be either homogeneous or heterogeneous.
- The moment we say it as homogeneous, it is of the similar type, and when it is an heterogeneous, it could be of different types.
- Lists can be accessed using indexing operation.
- Lists can be.
- Elements of a list can be accessed using indexing operation.
- Request you to see to the video on indexing operation.
- Indexing is also called as a subscript.
- And indexing always starts with zero.
- indexing always starts with zero and this that's the reason because of this particular reason it is also called as zero based indexing.
- And followed by a list can grow or shrink in the sense you once the list is created you can add or also you can remove the elements.
- List is an iterable.
- In this case it is eager.
- I the list is eager in nature.
- I can have a nested list, a list within one list.
- And I can assign one list to the another list.
- In such case both the list would be pointing to the same list.
- I can also create a sublist by making use of an operation called as a slicing operation.
- Please also look into the video of slicing operations.
- And Python provides you so many number of built-in functions where you can manipulate the list, you can work with the list, you can modify the lists.
- Before me going and executing the things, let me quickly iterate over the PPTs that are there.
- I'm going to be demonstrating it to you, so do not worry.
- And as I said you, there are so many operations that are available for us in order to perform the operations which we are going to be also seeing.
- Other operations also include concatenation, sorting.

Example or application from the lecture

- Before we discuss to find out the characteristics of a list, let's try to find out why do I really need a list.

- Okay, we are gonna be looking into each one of these attributes with respect to a particular example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000002 · EV0000003 · EV0000004 · +1 source units in pack.json

Core explanation · 05:00–09:59

Let's say L S T is equal and twenty thirty forty fifty.

Key details

- Linear ordering means.
- Just look at this and say print list.
- I'm trying to do the same thing here.
- I can I'm just getting my execution part ready.
- I will do all of them at the end I will run so I will say print of what is the length of the list.
- This is the first operation that I want to do.
- List provides there are some common functionalities that we can do with the list.
- Explain it to you in an easier way.
- And then again I will come back as and when it is required for us.
- I will say list is equal to ten comma, twenty comma, thirty, forty and fifty.
- Only difference between interactive and your what is that you have to give state.
- The length of the list is five in this case.
- It tries to find out what is the minimum element that is present.
- I can also perform something called as a sum operation on the list.
- it which adds every element in the list and gives you.
- List plus list.
- In this case, plus is called as a concatenation operation.
- Concatenation operation merges two lists and returns you a single list.

Example or application from the lecture

- When I say linear ordering, my input, the way in which the input is given and the way in which the output is given is same.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000006 · EV0000007 · EV0000008 · +1 source units in pack.json

Core explanation · 10:00–14:59

concatenation operation merges two lists and it returns you a single list.

Key details

- And there is something also called as a repetition operation.
- That's called as a repetition operation.
- Multiply a list and number of times.
- Values have been repeated.

- List elements can be duplicate in nature.
- And see I should have taught you how to create a list.
- Nevertheless, we will do it now.
- Creating an empty list is with the help of an open bracket and a close bracket.
- L S T is equal to L I S T of this.
- And I'll see what is the type of list.
- It is still a list itself.
- In this case, list of this is called as a constructor.
- It's called as a constructor.
- And in this case, the list is called as an empty list.
- How do you create a list?
- Make use of an open and closed brackets in order to represent.
- I can also create a list with the elements where I have already demonstrated.
- And let me s check with the list.
- At this point list is empty.
- This is my list.
- What and all I can do with this particular list.
- As list you can see there are n number of elements, but they are all belong to or they all have a similar name called as LST.
- But LST has a so many number of elements that are present.
- and access every element present in a list.
- How will I access the elements present in the list?
- I can make use of an indexing operation.
- As mentioned earlier, please look into my indexing video.
- as indexing starts from zero, it is also called as what?
- Zero based indexing.
- Called as zero based indexing.
- This is fine.
- Lists also supports slicing operation.
- List also supports a slicing operation.
- Again, see to the video on slicing.
- I can just mention from one and I will not mention anything to stop.
- This is a start value and this is a stop value.
- Okay, so I will say one to end which means start from a index one and go till the index end which is 50 in this case.
- This is fine.
- From three, start from one and stop at four.
- The moment I give four, which means four minus one because one less than the existing one.
- That from one go till three and one and three the values that I'm telling are the indexed values.
- They are the indexed values.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000010 · EV0000011 · EV0000012 · +1 source units in pack.json

Core explanation · 15:00–19:59

And I will try to find out what is the length of my list.

Key details

- Length of my list is five.
- All the values of a list are of same type.
- And then followed by a string.
- In this case list L one is called as heterogeneous in nature.
- The reason is it has an integer type, it has a floating type, and it also has a string type.
- Typo mistink in terms of the string, never mind, do not worry about it.
- Hope it is clear to you.
- How will I do it?
- I'll say DIR of a list to find out what functions does the list have.
- one more thing.
- If I want to find out or access fifty, what do I have to do?
- I will say list of minus one.
- there are many things that I've worked on the indexing topic.
- Please look into the video.
- Do with a few of the operations quickly.
- When I say append 100, it always appends the value to the end.
- You can see that the value hundred is present.
- I can also insert an item at a particular position.
- First parameter is the location that you need to specify and the second parameter is the value of which you want to insert.
- List is this and as I mentioned zero is now being replaced by two hundred.
- So hopefully you have understood the append method followed by the insert method.
- See, it's popped out the last element.
- Again, pop out one more, so it will pop out fifteen and so on.

Example or application from the lecture

- And if you can see the example says that a list has some integer values.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000014 · EV0000015 · EV0000016 · +1 source units in pack.json

Core explanation · 20:00–24:59

If nothing it represents it removes an item from the left side right to left, and if I say pop without any argument, it always removes the last item.

Key details

- And if it is pop with a particular argument.
- In this case the argument is representing the index value.
- I will remove the value hundred.

- One second, just wait.
- Previously I had removed the value 100 and I am trying to remove it.
- 10 is also not there.
- And when you remove the elements gets removed.
- As previously seen, if you're trying to remove a value which is not present, then it throws you an error.
- I'll say remove thousands.
- Okay, I cannot remove because the value thousand is not part of the list.
- List X does not contain this particular Y.
- I can also do this.
- This is my new list.
- List is 20 present in list?
- Answer is yes.
- Is 200 present in list?
- Is 200 not in?
- 200 not in the list.
- Yes, it is still true because it is not there.
- Index value we have already seen.
- Did we check with the index value?
- Just give me time.
- No, pop append insert.
- I will also try to add few values to the list.
- List.append of four hundred.
- I can also do list dot index of twenty.
- What is the index value of 20?
- 20 is present at the zeroth location.
- Two hundred and five, some number, some random number.
- Is two not five present in the list?
- No, it is not present, so it says it is an error.
- I'll say list dot count.
- How many times a particular element is present?
- If something is not present from where it can give you.
- And I can also say list.sort.
- There are some programming examples that we have to work with respect to the lists which I'm going to be doing in your coming classes.

Example or application from the lecture

- Let's look into what is our lists.
- Suppose if I say.
- For example, 200.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000018 · EV0000019 · EV0000020 · +1 source units in pack.json

Core explanation · 25:00–29:59

Hope you have understood.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000022 · EV0000023 · EV0000024 · +1 source units in pack.json

3 Dictionary

Study focus: Dictionary. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000026 · EV0000027

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Let me repeat, a set is an unordered collection of unique elements with zero or more elements present in it.

Key details

- I'll say S is equal to this.
- open curly breeze and closed curly breeze.
- And I want to find out what is the type of yes.
- And if I say what is a type of S, but basically say it as a dictionary.
- So which means my dictionary as well as set, we've in order to create them, we are going to be making use of curly braces itself.
- I need to work out what the set.
- Find out what is the type of S.
- But if your intention is to create an empty set, then make use of your constructor called as set in order to represent an empty set.
- Go to be set.
- He is called as a constructor.
- You can use constructor in order to create an empty set.
- Has got these some attributes.
- I'll be using a conventions yes, s1, s2, s3 in order to represent it as a set.
- The first attribute is that set elements are unique in nature.
- Set elements are unique in nature.
- What does it mean?
- I'll say two, one, four again, and a three again.
- And if you can see I've got one, two and three and four.
- Okay, so which means elements in a set are unique.
- The elements are hashable in nature.
- Elements are hashable in nature.
- That's all we need to know at this particular point.

- Fine, if it has an hash value which never changes during its lifetime.

Example or application from the lecture

- So let's create our first set.
- At this point, for our level, we just need to understand that hashing is a mechanism used to convert a given element into an integer.
- Hashing is a mechanism used to convert a given element into an integer.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000026 · EV0000027 · EV0000028 · +1 source units in pack.json

Core explanation · 05:00–09:59

Fine, which also means that it needs to have a function called as underscore hash underscore as one of its method.

Key details

- Data structure having this particular method, then we call it as what it is hashable in nature.
- And if you can see and hashable method if it is available then we basically call it as what?
- It is uses a mechanism called as hash method.
- It is clear to you it's a mechanism that is used in order to.
- The ones that are not hashable are list, dictionaries, and all of them.
- Sorry I'm creating a set.
- I'll say set.
- I'll say L of S.
- This is five.
- What is the minimum element in this S?
- Minimum element is one.
- And I can I do max of yes.
- Max is element is five.
- I also see some of yes.
- And I'll do one more thing.
- I'll create something like this.
- Yes one is equal to hundreds twenty two.
- When I say S1, if you can see, if you can see, it's evident that set elements are unordered in nature.
- The order of the set cannot be determined.
- An important attribute is that the set elements are unordered in nature.
- We can never assume the order of elements.
- The reason is the way in which I have entered and the way in which it is present internally is different.
- because of which because of which I cannot do something like this.
- I cannot do because it does not support indexing.
- The main reason is the set elements are unordered in nature.

- I'll see S1.
- dot add and I'll say two hundred.
- This is fine.
- I said s one dot add which is two hundred.
- You could see S1 where 200 is being added.
- The moment you say add, it adds a well.
- Where does it add a value?
- We never know because it is based on the concept of hashing, which means sets are mutable in nature.
- sets are mutable.
- I can change the contents after it has been created.
- Remember, please always remember.
- Set is mutable, but it elements are.
- Its elements are immutable.
- Set is mutable in nature, but its elements are immutable.
- It means that let me create this.

Example or application from the lecture

- Convert a given element into an integer.
- Let's go ahead with the next one.
- Or before that let's find out what are the common operations that we could do it on this set.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000030 · EV0000031 · EV0000032 · +1 source units in pack.json

Core explanation · 10:00–14:59

One, two, three, and I'll create a list within this.

Key details

- So let me do this and it says unhashable type list, which means my set has a whole is mutable but its elements are immutable, which always requires you to have the values that are hashable objects.
- Set is mutable, it is unordered, duplications are not allowed, it is unique in nature, okay.
- It does not support indexing.
- As it does not support indexing, I cannot also perform slicing operation.
- This is my S L C for I N S print I.
- And see I've got the values, which means set is iterable.
- Set is iterable.
- What is the use of a set?
- Sets are basically used in order to remove the duplicate elements, which in turn allows us to have or find the unique elements that are present.
- Sets are very useful in order to remove the duplicate elements.
- The moment I remove the duplicate elements, sorry, duplicate elements, I'm going to be having the unique elements that are present.

- I can also compare.
- Two I trables for common elements or common differences.
- Is this clear?
- We will also be working on this particular concept.
- And there's one more thing that I want to.
- I'm talking about a membership operator.
- Returns true if the element is present in set.
- returns false if the required element is not present in a set.
- Okay and I can also do not in it's true because it is not present in the set.
- I'll say D I R of sets.
- And you can see these are the functionalities that are available.
- I'm going to be showing you a few of them.
- The rest of them are left for you to understand.
- In order to do, I will try to make use of two different sets, and the values of these sets are.
- Five and six.
- And I will have my S two being.
- Six, seven, eight.
- Want some common elements also, so I will retain five because I can have it in any order, so order doesn't matter.
- I'm trying to perform concatenation operation.
- Set and set concatenation operation is not supported.
- Can I do this?
- S1 plus two.
- Says still concatenation operation is not supported.

Example or application from the lecture

- Let's find out.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000034 · EV0000035 · EV0000036 · +1 source units in pack.json

Core explanation · 15:00–19:59

Is it fine?

Key details

- I'll say S1 [ASR low confidence; check audio] dot union of estop.
- I'll say s1 dot union of s two, which means I am combining the values of s one and s two.
- Returns the union of sets as a new set for us.
- I can also say this as one, you can also make use of this particular symbol in order to represent it as what?
- The union operation.
- I can also do something like this.
- S1 minus S2, the ones that are present in S1 and the ones that are present in S2 have been removed from S1.

- So according to this one, two, three and four because five six is here, five six is also here.
- The other way is also true for this.
- I can also say S1 minus S2.
- Remove the ones that are.
- There in both S1 and S two and give the elements of S two only.
- And if you can see even though we said seven, eight, nine and ten, the order has not been maintained.
- [ASR low confidence; check audio].
- This is my S1.
- We will see S1.Remove.
- S1 dot remove.
- Which one shall we remove?
- We'll remove three from here.
- Is this fine?
- What does a remove operation will do?
- It removes the specified item in a set.
- If an item is not present, okay, now we have already removed three.
- I'm trying to remove three again.
- It throws us an error.
- Otherwise it throws us an error.
- Also do discard.
- Discard four is what we will do.
- We'll try to find out.
- What does this discard will do?
- I'll say yes one.
- It has also removed.
- Remo also did the same thing for us.
- I will try to remove the same item again.
- And in this case, it does not throw me an error.
- Removes a particular element.
- Discord also does the same thing.
- If an item is not present, remote throws us an error, but whereas discard does not throw us an error.
- And S1.pop is an operation.
- It pops out some element.
- In this case it has popped out one.
- Let me pop out one more.
- It has popped out five, in this case it has popped out six, and so on.
- Removes an element or an item in a set.
- In this case, we are lucky enough that the pop operation is removing elements one by one.
- Is it guaranteed?
- No, it is not guaranteed.
- The reason is sets are unordered.

Example or application from the lecture

- Let's find out what does it do.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000038 · EV0000039 · EV0000040 · +1 source units in pack.json

Core explanation · 20:00–24:59

There are a few more things that you can always work out with the set.

Key details

- I've just shown you a few of them.
- I request you all to try out be the other ones because they're all self explanatory.
- Is this fine?
- Symmetric difference, symmetric difference update.
- Wherever you have something called as an update.
- It always does the changes to an existing set.
- Please remember, whenever you have an update, operation is performed at the same time where it is being done, it is going to be done to that particular set itself.
- Is this fine?
- I request you all to try out with the other functionalities that are present.
- keep performing the operations.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000042 · EV0000043 · EV0000044 · +1 source units in pack.json

4 Files

Study focus: Files. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000046 · EV0000047

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Command line arguments.

Key details

- My main intention is that I want to perform addition of two numbers addition of two numbers using command line arguments.
- And this is my Y, but my argument in this case could be two.
- Sum is equal to X plus Y.
- And I want to previously let me print the values how it was initially.
- Anyway, I don't need them.
- I will be doing it.

- I want to find out what is the initial value and how will I give the how will I get the final result.
- Fine, and when I do this, just look at what happens.
- It says index list index out of range.
- The reason is when I am passing an input as a command line arguments, my input also should be a part of the command line itself.
- This is R g of zero, this is R g of one, and R V of two.
- My final output I'm expecting is a fifty in this particular case.
- We have successfully executed giving an input through the command line arguments.
- Previously you would have already seen the other way of giving inputs.
- You want it to be an integer, so you say it as an int of inputs.
- I will give an input called as twenty, the other one is thirty again.
- The inputs that we are trying to give is very minimal.
- humanly impossible to specify hundred different values on the command line.
- The reason may be we may have some errors while giving in the sense I may end up just giving 99 or I may end up giving one out one also.

Example or application from the lecture

- So take a simple example in this case.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000046 · EV0000047 · EV0000048 · +1 source units in pack.json

Core explanation · 05:00–09:59

But in both the cases it is not feasible.

Key details

- there may be a situation where you want to work with a particular application where the amount of data that is required is more for you.
- The larger data is required.
- Hopefully it is clear to you why we need files.
- Data is persistent, even after your program terminates, the data is persistent.
- You're going to be working with larger amount of data.
- A file is some information or data which stays in a computer storage device, especially in computer memory.
- One is called as a binary, and the other one is called as a text file.
- Binary files, it contains the sequence.
- And very difficult for human to read.
- At the same time, when you have a text file, you can store data in the form of alphabets, digits and other special symbols.
- In this session we are going to be working with the text files.
- When I say we will be working with the files, a file may be used for a reading, or it can be used for writing.
- May be used for reading or it can be used for writing purpose.

- When I see a file, I can use an existing file, or I can create a new file, or I can overwrite an existing file.
- You can make use of an existing file, so overwrite, both means the same in this case, or you can create your own files.
- Reading and writing.
- You may also specify reading as R.
- Writing as W, reading as R and writing as W.
- Where R and W represent modes.
- R and W represents modes.
- There are other modes also, but they are out of scope for our syllabus, so we are going to be discussing only a read and write operation.
- One is open a file.
- Reading or writing operation.
- At the same time you will also have to close the file.
- You will also have to close the file.
- Hope it is clear.
- If a file that I want to open or before that, in order to open a file, I will make use of a function called as open.
- I want to open it only for the reading purpose.
- Now in this case if t2.txt is in existence, which means if it is present, then open function will return an object.
- Open function will return an object for this particular.

Example or application from the lecture

- And at the same time, it's chances of errors are very less when input is given through a particular file.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000050 · EV0000051 · EV0000052 · +1 source units in pack.json

Core explanation · 10:00–14:59

F is a file object.

Key details

- On success, so it gives me if t2.txt does not exist, it throws me an error.
- Let me repeat.
- If one success, it returns a file object.
- If it is not present, it throws me an error.
- It's still executing.
- Maybe I haven't commented on it.
- We don't need them.
- In this case, what am I trying to do?
- I'm trying to open a particular file.
- If it is not present, it throws us in.

- File not found error has been thrown.
- When you create a file, make sure your dot txt and dot p y resides in a same part.
- resides in a same path.
- Otherwise you will have to specify the entire path where your text file is available.
- In and both my.py files and both.txt files are present on the desktop at this particular point.
- What is that output?
- And it says it is success because twin.txt was already available for us, so and I was successfully able to open the text file.
- This is how we try to open a particular file.
- There are different ways to perform a read operation.
- One is the first one is by making use of something called as read line.
- There is something called as a read line.
- The other one is read lines.
- And the last one is actually read.
- Read comes with and without parameter.
- Read comes with and without parameters.
- This is Python files dot txt and dot py should be in the same location.
- This is what the contents of my text sir.
- Yes, T1.txt file.
- Let me execute again.
- I do not want this.
- in read mode.
- We are asking you to open.
- I want you to open it in a read mode.
- And I want to read.

Example or application from the lecture

- Let's not worry about it.
- Let's not worry.
- Let's go back here and we will perform these operations on that particular text file.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000054 · EV0000055 · EV0000056 · +1 source units in pack.json

Core explanation · 15:00–19:59

If you can see it has read the first line from the file, but there is a huge gap between this and our prompt.

Key details

- Is this fine?
- The reason is the function read line, it reads a line from a file, in the sense read line is reading this from a file along with reading, it also includes a new line character.
- It also includes a new line characters at the end of the line in a file.
- Along with that.

- Along with that, you do have a print statement.
- By default, print statement already has a new line within it, because of which I am getting two lines as an extra line here.
- No, we do not want any extra lines to be available.
- I need to get rid of this particular line.
- read line dot strip.
- Make sure you assign it to the same variable.
- A sign it to the same variable.
- Okay, if you can see we have got rid of one line from this to this because print statement already has a new line because of which we are getting this particular line.
- If you still want to get rid of that also, what you can do is you can specify.
- And is equal to nothing.
- We will not have that extra line also in the output.
- So similarly we will go to the next one and I will just copy paste a few things so that I want to make my life easy.
- In this case I will use read.
- Read is a function which reads your entire file.
- Read is a function which basically reads your entire file.
- And see the entire file has been read.
- Or if you are interested, you're also free to read only certain bytes of information from your particular file.
- This is also possible.
- At the same time, you can also read the entire file.
- There are other ways of reading a file where you can iterate over using your while loop and then get each line using your read line and print the same.
- any extra lines you can make use of your strip function and get rid of a particular line.
- Okay, so the next one is we went through read line, we also had a read option and there is one more which is called as read lines.
- I will use these two lines.
- I don't want the previous one, so I will comment it.
- A read lines reads the whole file into a list.

Example or application from the lecture

- Let's look into this now and find out the difference.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000058 · EV0000059 · EV0000060 · +1 source units in pack.json

Core explanation · 20:00–24:59

Please remember read file reads the entire or a whole file into a list.

Key details

- Where each element of the list is a line of a file.
- Where each element of a list is a line of a particular file.

- Print read lines.
- These are the few things that we have learned.
- Okay next one is to perform writing operation to a file.
- write to a file.
- Some file where we will say or I will make use of the same file t1.txt itself I will make use.
- But I have to open it in a write mode.
- The moment you open it in a write mode, your previous content will be lost.
- If you can remember my previous contents were this is Python files, dot txt and dot py should be in the same location.
- What we will do is we will try to print.
- To which one?
- To a file.
- Even.txt whose handle is F.
- If you remember, I just want to show it to you.
- When we did your print statement.
- When we did your print statement.
- File is equal to system dot standard output.
- You wanted your output to be available.
- All this time your output was available on the standard output.
- But right now, whatever you are writing, you want your output to be available in a particular file, which is t win.txt.
- File is equal to F.
- Exits I'll say python files.py and it has successfully executed.
- We had opened it long back, so I just closed the file and then opened it for viewing the content.
- we strictly tell you to just make use of a print statement in order to write it to a file.
- Hope you have understood the concept of files.

Example or application from the lecture

- In the notes, we have also given you using a write statement, okay, write as a function.
- So in the coming sessions we will also try to work with a few example programs.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000062 · EV0000063 · EV0000064 · +1 source units in pack.json

5 Function_Call_Definition_Variable_arguments

Study focus: Function_Call_Definition_Variable_arguments. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000066 · EV0000067

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

I am Nita Anjaikan from the Department of Computer Science and Engineering.

Key details

- In this lecture, we will look at the concept of functions.
- We will look at how to define functions, what exactly functions are, and how we could call or use them.
- It may seem like a bunch of jargon to you, but let me put it in a simpler way.
- When you have a set of code that say would be used multiple times in your program, rather than copying or repeating those lines of code multiple times, I would rather club all those reusable lines together, put a name for it.
- Functions ideally would take an input, performs a specific set of operations, and computes and results gives you an output.
- We are actually familiar with this concept as we have seen in the first unit when we talked about your input and output functions.
- It takes an input, processes your input, does a couple of steps, and then returns an output.
- There are different kinds of functions available in Python.
- You have built in an existing functions like the print and input function, and there are user defined functions which as a programmer we can create based on the needs of our application.
- What are the advantages of using functions?
- They are, like I've already said, it reduces the duplication of code by just call the name of the function.
- It supports modularity, which means that every feature of the application that you build could be a function.
- It improves the clarity of code because it's so neatly structured as you will see in our examples that you can easily debug the code.
- It promotes reuse, it enhances readability, and the debugging and maintenance also becomes easier because when that there is a problem with a specific feature, you can directly jump to the lines of code where the function is defined and debug the code appropriately.
- So in our programming term when we say function definition, it means that we're going to try and create the function.
- Followed by the name of the function.
- A function name is an identifier, which means it should start with alphabets, then followed by a number or letter in English, or a digit, followed by a pair of round parentheses and then a colon.
- It is to give an input to the function if required.
- We will look at all of that in the upcoming lectures.
- Once we have and the leader with a colon, you can then use an indented block called the suite, which can have any valid statement in Python.
- It could be one or more statements, or it could also be another function, as we will see.
- As you can see, there is define the name of a function followed by a pair of parentheses and the parameters are optional.
- Then you have an indented block which would be the suite of the program.

- We must note that a function must be defined before we use or call the function.

Example or application from the lecture

- So first let's explore at what a function really is.
- there would be a leader and a suite because we are going to have multiple lines of code that are going to be clubbed and given a single name.
- So let's just look at an example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000066 · EV0000067 · EV0000068 · +1 source units in pack.json

Core explanation · 05:00–09:59

I know this is a redundant example, but it's just to show you how exactly it works.

Key details

- Every leader must end with a semicolon.
- Command prompt and I'm going to execute functions.py, which is the file in which I have written the code.
- You can see there is no output because all I have done is create the function.
- I have not brought the function to life.
- Followed by what we would call a function call operator.
- Invoke or we would activate the function.
- So when a function is defined, as you saw that when I executed, I did not get any output because the header or the leader is processed.
- By using the function call.
- At that point of time the control is shifted to the entire function block.
- The function is processed.
- And there was no output.
- It goes on to the next set of statements.
- Before display, I'm going to say print before function call.
- And after display print after function call.

Example or application from the lecture

- So let's look at another example where I'm going to like we've shown in this particular PPT, I'm going to create the same display function and have multiple print statements to show you what exactly would happen if I have other statements and what exactly the name of the function actually means.
- So this example is just to prove that.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000070 · EV0000071 · EV0000072 · +1 source units in pack.json

Core explanation · 10:00–14:59

The first two lines are not executed in the beginning.

Key details

- It's always going to be before and in between the before and after function call.
- You can see it says it is a function display at a particular location.
- This is because it's stored in the global memory, right?
- Which means your function is actually.
- just another object which has a property to be reused.
- What does this statement mean?
- Since the function is an object, I can assign one function object to another.
- I'm going to write display1 and the function call parameter.
- Heading over to the interpreter.
- So when you look at print display and print display one, you can see that both of them are functions at the same location, which means the name of a function is a function object or a variable which is basically a whose value is a function and that can be activated using a pair of round parentheses.
- Does this delete my function altogether?
- So if I were to now just call display in the next line, okay I'm going to save it.
- Head over to the terminal and execute it, you can say I get an error saying the name display is not defined.
- That's because I have deleted the display function here.
- After this I'm going to write display1 with the function call.
- Will this work?
- It works as expected.
- I can call it using either one of them.
- Associated with the function.
- It doesn't delete the function itself, which means I have display one also pointing to the same function.
- I hope you have understood the concept.
- In the upcoming videos, we will look at what exactly happens when a function is called, how we could give input to the function.
- and process these values as well.

Example or application from the lecture

- Let's look at it.
- Infer from this particular example is that display and display one are just names for that function object in the memory.
- So let's look at the next set of information in the upcoming class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000074 · EV0000075 · EV0000076 · +1 source units in pack.json

6 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets

Study focus: Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000174 · EV0000175

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Definition:- A set is an unordered collection of Unique elements with zero or

- more elements present with the following attributes.
- In Python sets are written with curly braces.
- ConstructorConstructor set()set() isis usedused toto createcreate anan emptyempty setset.

Slide 3 · Mutable: Modifiable

- Unordered – Order of elements in a set need not be same as the order in which we add
- NotNot indexableindexable - CannotCannot accessaccess itemsitems ininin aa setset bby referringreferring toto anan indeindex, sincesince setssets areare
- Check for membership using the in operator is faster in case of a set compared to a list,

Slide 4 · Elements are unique – does not support repeated elements.

- Hashing is a mechanism to convert the given element into an integer.
- An object is hashable if it has a hash value which never changes during its
- lifetime (it needs a __hash__() method).

Slide 5 · #set1={ [1, 'a', 'hi'], 2, 'hello', {3, 4.5, 'how r u' } }

- #set2={ (1, 'a', 'hi'), 2, 'hello', {1:'hi', 2:'hello', 3:'how r u'} }
- set3={ (1, 'a', 'hi'), 2, 'hello', 5.56, 3+5j, True }
- #{True, (1, 'a', 'hi'), 2, 5.56, (3+5j), 'hello'}

Slide 6 · Common methods of Set

- There are number of functions built into Python that takes set as the

Slide 7 · Specific methods of Set

- The dir() function returns all properties and methods of the specified object.
- o ['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__',
- '__init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__',

Slide 8 · Methods in detail

- len() Returns the number of elements in a >>> set1={22,33,11,55,44,120}
- max() Returns the element from Set with >>> set1={42,73,25,75,64,145}
- min() Returns the element from Set with >>> set1={25,43,21,66,45,120}

Slide 9 · Common methods of Set

- sorted() Returns the sorted sequence or sorted >>> set1={22,33,11,55,44,120}
- collection in the form of list. >>> sorted(set1)
- [11, 22, 33, 44, 55, 120]

Slide 10 · Common operators applied on set

- in a set that contains items of the two sets.
- Repetition operator(*) – Throws an error as set has unique elements.
- TypeError: unsupported operand type(s) for *: 'set' and 'int'

Slide 11 · Common operators applied on set

- Why are we using the |= operator here?
- We used the + operator for lists and tuples, so why not use that here?
- What's the difference between +, |, and |=?

Slide 12 · Common operators applied on set

- Membership operator(in) – returns True if a sequence with the specified value
- Membership(not in) operator- Returns True if the element is not present in
- the set, else return False.

Slide 13 · Specific functions of set

- union()- Return the union of sets as a new set. (i.e. all elements that are in
- {1{1, 22, 33, 44, 55, 66, 77, 88, 99, 10}10} {1{1, 22, 33, 44, 55, 66, 77, 88, 99, 10}10}
- intersection()- Return the intersection of sets as a new set. (i.e. all elements

Slide 14 · Specific functions of set

- difference()- Return the difference of two or more sets as a new set.
- >>> s1={1,2,3,4,5,6}
- add()- adds an element in a container set.

Slide 15 · Specific functions of set

- symmetric_difference()- Return a set with the symmetric differences of two
- >> s1={1,2,3,4,5,6}

- remove()- Removes the specified item in a set. If the item to remove does

Slide 16 · Specific functions of set

- discard()- Removes the specified item in a set. If the item to remove does
- pop()- Removes an item in a set. Sets are unordered, so when using the
- pop() method, doesn't know which item that gets removed.

Slide 17 · Specific functions of set

- update()- updates the current set, by adding items from another set (or any
- other iterable). If the element is present in both the sets, only one
- appearance of element present in updated set

Slide 18 · Specific functions of set

- difference_update()- Removes the items that exists in both sets.
- Symmetric_difference_update()- Updates the original set by removing items
- that are present in both sets, and inserting the other items.

Slide 19 · Specific functions of set

- issubset()- Returns True if all items in set 'x' are present in set 'y'.
- >>> y = {"f", "e", "d", "c", "b", "a"}
- issuperset()- Returns True if all items in set 'y' are present in set 'x'.

Slide 20 · Specific functions of set

- isdisjoint()- Returns True if no items in set s1 is present in set s2.
- clear()- Removes all the elements in a set.
- >>> x = {"f", "e", "d", "c", "b", "a"}

Slide 21 · Example code 1:

- Display the elements of a set one by one in separated by a space between
- a = { 10, 30, 10, 40, 20, 50, 30 }
- Output: 40 10 50 20 30

Slide 22 · Example code 2:

- Create a set of numbers from 2 to n.
- Steps: create an empty set and add the elements to it.
- Output: {2, 3, 4, 5, 6, 7, 8, 9, 10}

Slide 23 · Example code 3:

- Program to check whether a set is empty or not.
- if len(s1)==0: if(s1):
- Note: Empty data structure is False

Slide 24 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000174 · EV0000175 · EV0000176 · +69 source units in pack.json

7 Source slide deck: JSON

Study focus: Source slide deck: JSON. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000306 · EV0000307

SHORT-NOTE RECAP

Study notes · Source slide deck: JSON

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · Solving- UE26CS151A

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 2 · Introduction to JSON

- It is a text-based format for representing structured data as
- It's lightweight as it stores info in text "courses": ["PCPS", "DSA"]
- data(including virus) making it safe to It is used in Web APIs,

Slide 3 · Introduction to JSON

- Only 6 data types exist in JSON. Every one has a direct Python equivalent, which is
- why json.load() can hand you back native Python objects with zero manual
- JSON type Python type Notes

Slide 4 · Syntax rules in JSON compared to Python

- (Python dict syntax, single quotes) is not valid JSON
- Python tuples/lists tolerate this, JSON does not
- No comments allowed in JSON — unlike Python # or JS

Slide 5 · Json module and functions

- import json There are 4 fundamental json
- functions used to serialize orPython's standard
- json.loads(str) json.dumps(obj) returns

Slide 6 · Json.dump()

- with open("student.json", "w") as file:dump() returns None
- json.dump(student, file, indent=4)Because it streams the
- It takes your Python data (like a If the program crashes mid-write

Slide 7 · Json.load()

- It reads a static .json file from your
- with open("data.json") as f:format into dynamic, manipulable
- Python objects like dictionaries(for data=json.load(f)

Slide 8 · Json.dumps()

- Because HTTP APIs and internet json_str = json.dumps(data,
- mechanism for packaging complex print(json_str)
- backend data for network print(type(json_str)) # <class 'str'>

Slide 9 · Json.loads()

- data = json.loads(json_str)within the program
- When your backend receives an print(type(data)) # <class 'dict'>
- json_str above is a Python stringstring. loads() is the tool that

Slide 10 · Common errors and good practices

- and lists. It has no idea what a Python date, set, or custom object is. If you try to save them directly,
- your code will crash. You must simplify them first: turn dates into text strings, and turn sets into
- alone and keep it readable, you have to tell it ensure_ascii=False when you save the data.

Slide 11 · What JSON is JSON vs Python — matching

- key-value pairs and lists. It looks a {} object → Python dict
- json.dump(obj, file) — write Python JSON keys must always use
- json.dumps(obj) — turn Python data last item

Slide 12 · Time to be smart!

- The Mission: Build a simple Flask or FastAPI server that acts as a digital barista. It must receive a raw text
- 1.Catch the Order (Deserialize): Create a web address (an endpoint) called /order. When the phone sends
- 2.Process the Data (Logic): Using Python, look at the dictionary. If a Latte costs $4, multiply that by the

Slide 13 · Department of CSE (AIML)

- Dr Manjula K- manjulak@pes.edu
- Ack: Teaching Assistant - Sudarshan G(CSE

EV0000306 · EV0000307 · EV0000308 · +36 source units in pack.json

8 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args

Study focus: Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000408 · EV0000409

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- When we call a function in Python, we can pass in two different types of
- or method which is preceded by a keyword (parameter_name) and an
- Note: If there are both keyword and positional arguments, keyword arguments

Slide 3 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Parameter names are used to pass the Arguments are passed in the order of
- argumentg duringg the call parameters.p The order defined in the
- Order of parameter Names can be Order of values cannot be changed to

Slide 4 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Function Call: Keyword and Positional Arguments
- print(“Hiprint(Hi, II am”name)am ,name)
- nameAge(name=“Rajeev”, age=26)

Slide 5 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Function Call: Keyword and Positional Arguments in Python
- print(“Difference between two numbers is=”,minus(X,Y))

Slide 6 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Function Call: Keyword and Positional Arguments in Python

- f1(3,5,x = {4,5},b = [12,7,8,4], a = 99.7) #TypeError: f1() got multiple values for
- f1(x = 9, y = 99,2,3,b = 22) #SyntaxError: positional argument follows keyword

Slide 7 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Default parameters are always a part of the symbol table which is added
- during the leader processing phase. If the user did not send any argument,
- then this default parameter is used in the processing.

Slide 8 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CCBD & CDSAML, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000408 · EV0000409 · EV0000410 · +21 source units in pack.json

9 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args

Study focus: Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator.

EV0000432 · EV0000433

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- positionaliti l args andd KKeywordd args
- Department of Computer Science and Engineering

Slide 2 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- In the function definition, variables are specified which receive the
- arguments when the function is called or invoked. These are called
- When the function call is made, the control is transferred to the function

Slide 3 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- When a function performs a specified task, it may require some values to
- These values have to be provided when calling a function as input.

- These values have to be put within the pair of round parentheses in the

Slide 4 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- variable is within the file and outside that file too.
- Local variables: Variables created inside the function and accessible only to
- print("global first outside",x)

Slide 5 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Function: Global and Local variables
- Example 2: Global variable is read-only inside a function. We cannot modify
- the value of that variable inside the function directly.

Slide 6 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Example 3: Usage of global keyword
- print("global first outside",x)
- #If you interchange the first two statements inside the function, it results in

Slide 7 · Function Call: Arguments and Parameters

- Example 4:- Calculate the sum of two numbers given and return the
- print(“The sum of two numbers is = ”,output)
- The sum of two numbers is = 15

Slide 8 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Example 5:- Calculate the area of triangle given the dimensions.
- requires two arguments to be passed when the function is
- The arguments are constants in this example, 5 and 6.

Slide 9 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- When a function is called, it is imperative that the number of arguments
- passed MUST ALWAYS MATCH the number of parameters in the
- #r=product(5,6,2,7) #error : too many arguments

Slide 10 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Function Call: Categories of Functions
- 1. No arguments : No return value 1. No arguments : with return value
- print(a+b) return a+b

Slide 11 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Function Call: Categories of Functions
- 3. With arguments : No return value 4. With arguments : With return value
- def add(a,b): def add(a,b):

Slide 12 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Variable number of positional arguments: *arg [Any variable can be used with *]
- handles the variable number of positional arguments in the function.
- f1(2,1,6,4,9,7) #All three function calls are valid

Slide 13 · PYTHON FOR COMPUTATIONALPROBLEM SOLVING

- Variable number of keyword arguments: **kwarg [Any variable can be used with **]
- for i in kwarg.keys():
- for i in kwarg.values():

Slide 14 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CCBD & CDSAML, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000432 · EV0000433 · EV0000434 · +39 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Official unit catalogue	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/units.json	Official course/unit title and index.
Lists and Tuples transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Lists and Tuples transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Dictionary transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Dictionary transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Dictionary transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Dictionary transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Files transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Files transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Files transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Files transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Function_Call_Definition_Variable_arguments transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples (converted PDF; original hash f48314ea61e3a37dde957739f25e19751931c96d26a4c1d7f4015fc3e3fd15f1)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary (converted PDF; original hash e12e33099905b6cc61d27afa43ae9798feddcd77af96ff7b373169cdeaff78c2)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets (converted PDF; original hash d55f37758c0a92a8cfd16debf5d71e7fe5eb8f198e1c939871c68b44e0d77594)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files (converted PDF; original hash c60226e6bca6d6586282798100dc860deb793fc64d9dc613935de662682a2273)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: JSON	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: JSON	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for JSON (converted PDF; original hash 6d3e445fc984614d9591c7e9bf6f4fb66a9b7bce55ca051618d12a4126586a18)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call (converted PDF; original hash c658adc22df5ed0170035e56c1f1e3a699d4178f0fb43230ab11d8c0d18821b8)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args (converted PDF; original hash 792efdd1303b9d91389b09b4a2042f4c06dcc9c89cf41cf767c7af0de93e7e12)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args (converted PDF; original hash 14a7b0878d91cb0b8e9ab72ffa872ba4715bd95ed3834b9c977389c05def10f9)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 3 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00001	1 covered · 0 excluded · 0 gaps	catalogue: Unit title and index copied from the official units.json catalogue; human content review is pending.
SRC00002	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...
SRC00003	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00004	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...
SRC00005	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00006	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare...
SRC00007	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00008	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare...
SRC00009	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00010	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...
SRC00011	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00012	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...
SRC00013	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00014	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...
SRC00015	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00016	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...
SRC00017	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...
SRC00018	0 covered · 0 excluded · 16 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...

Source	Unit counts	Example decisions
SRC00020	0 covered · 0 excluded · 16 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00019	0 covered · 0 excluded · 16 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00021	0 covered · 0 excluded · 16 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00023	0 covered · 0 excluded · 16 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00022	0 covered · 0 excluded · 16 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00024	24 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00026	24 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00025	24 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00027	0 covered · 0 excluded · 20 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00029	0 covered · 0 excluded · 20 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00028	0 covered · 0 excluded · 20 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00030	13 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00032	13 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00031	13 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00033	0 covered · 0 excluded · 21 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00035	0 covered · 0 excluded · 21 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00034	0 covered · 0 excluded · 21 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00036	8 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00038	8 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00037	8 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00039	14 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00041	14 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00040	14 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

C Revision and retrieval practice